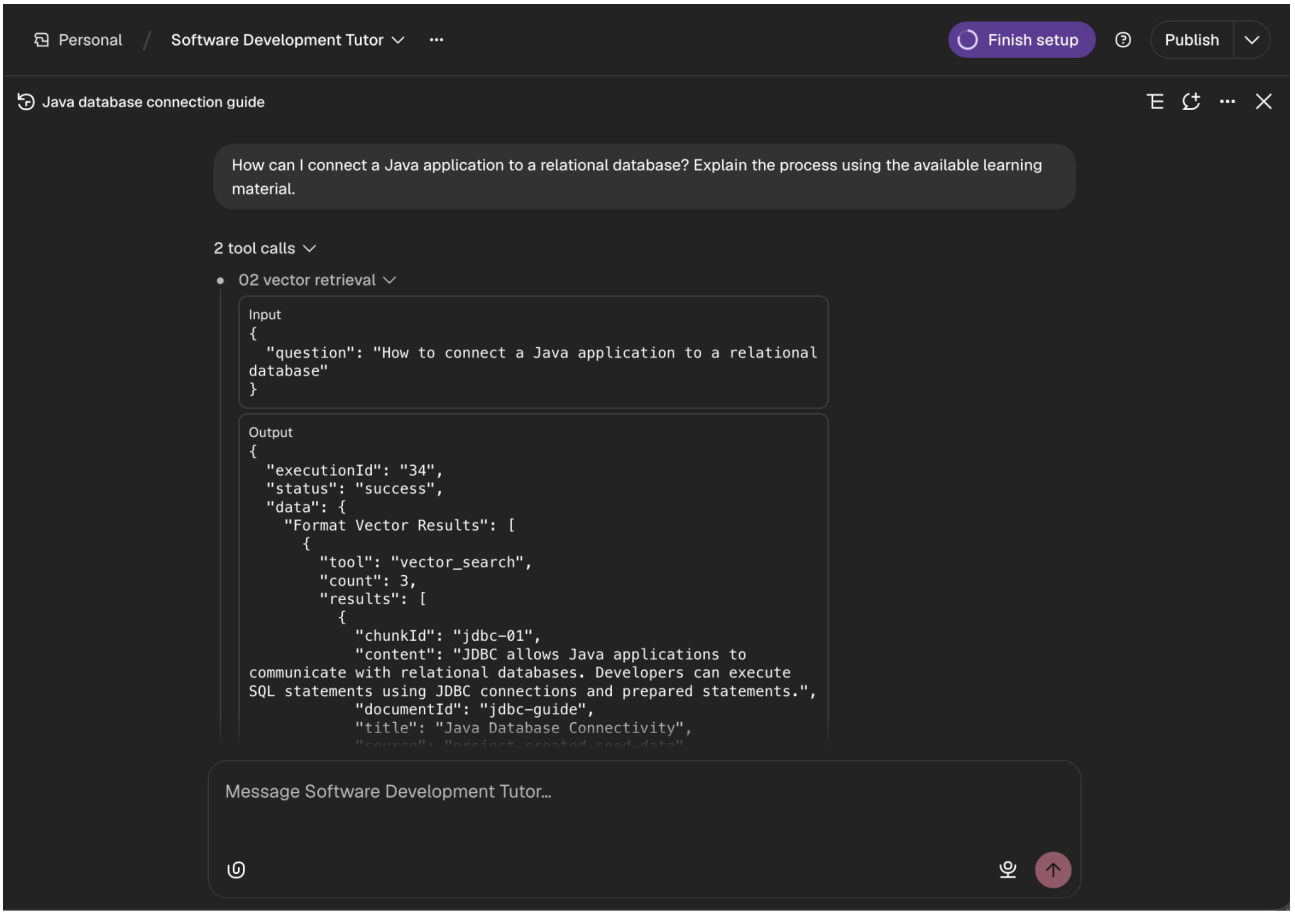
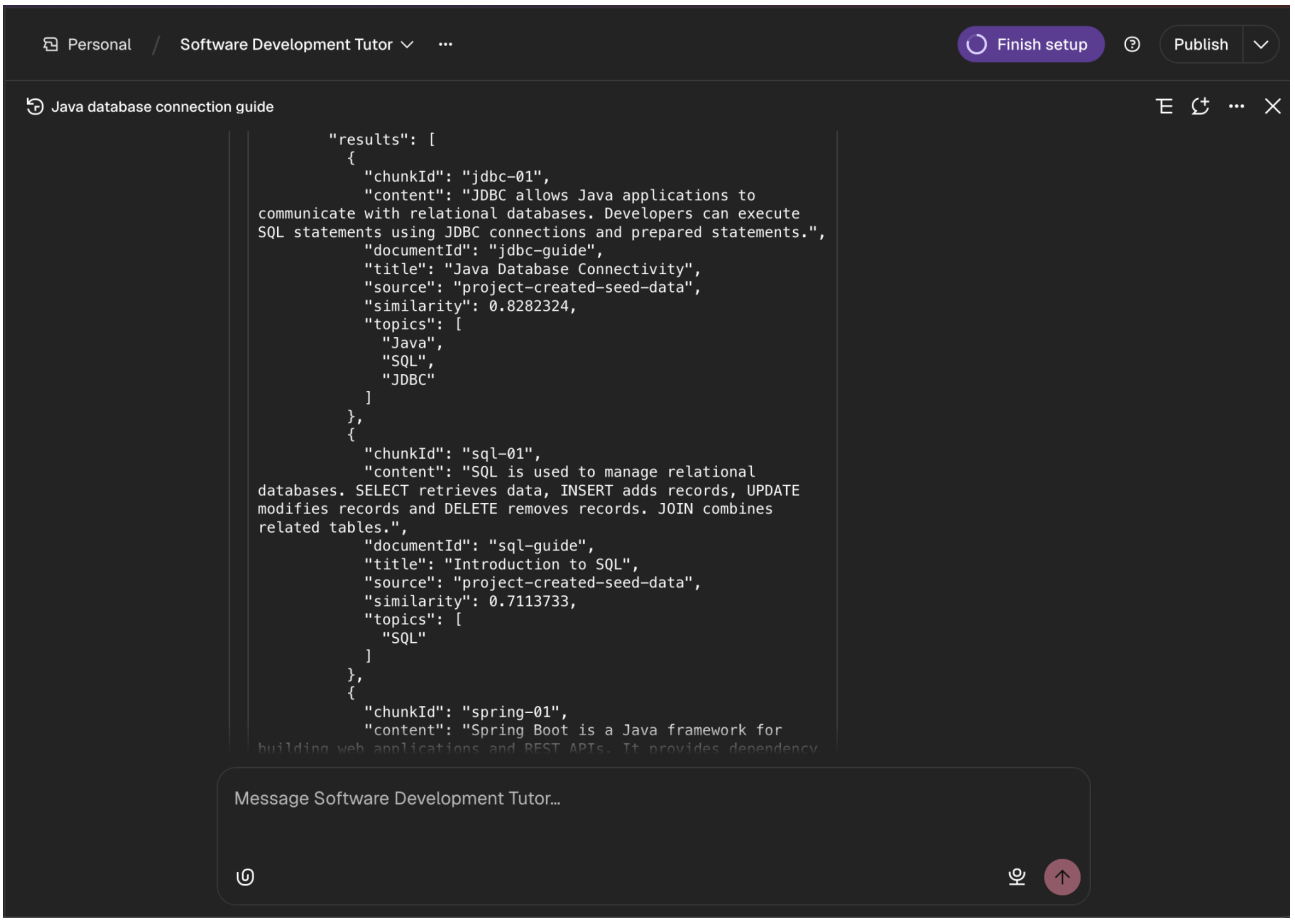
Test 1: Vector RAG

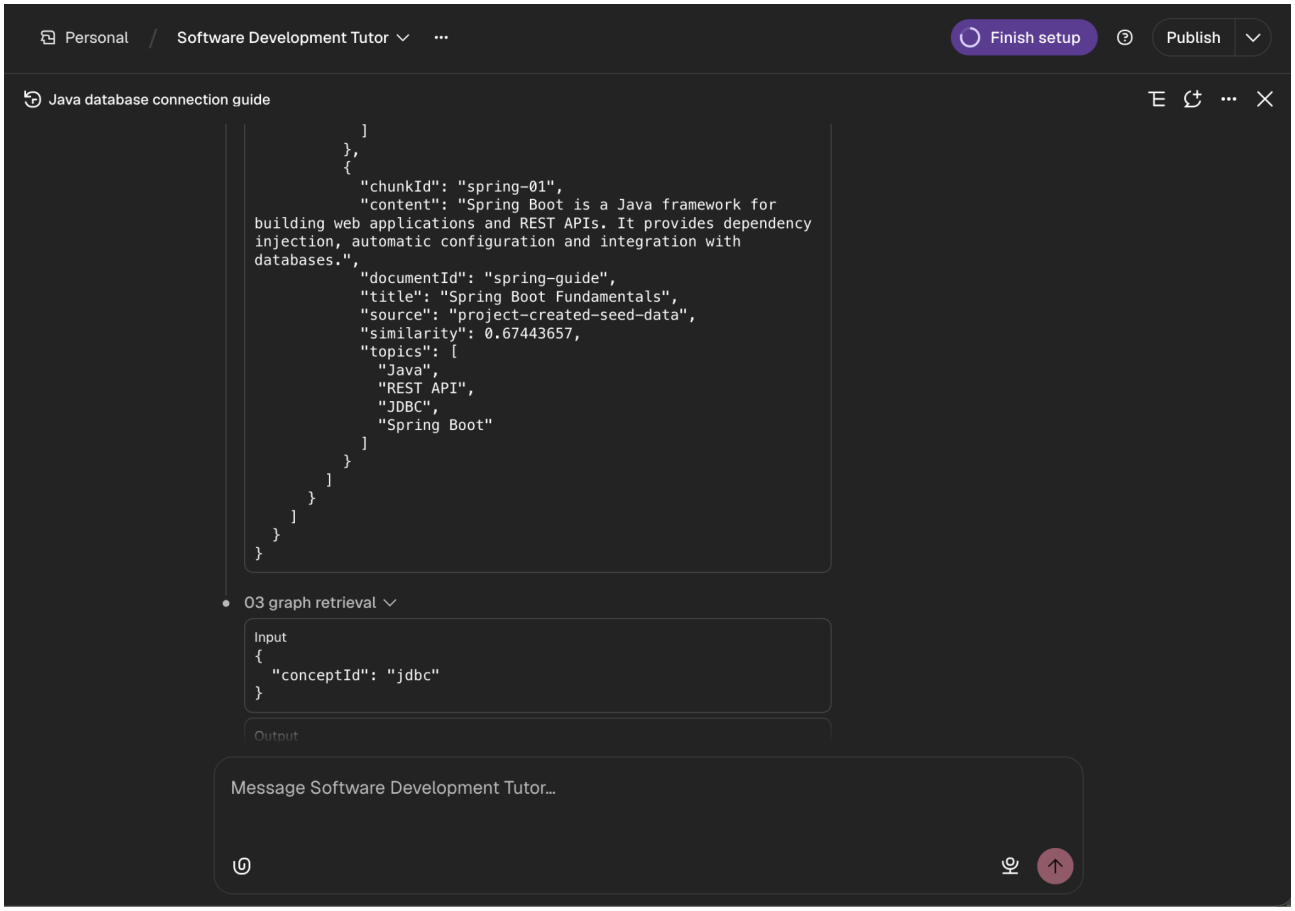
Vi undersøger, om Claude kan finde relevant undervisningsmateriale, uden at spørgsmålet nævner JDBC.

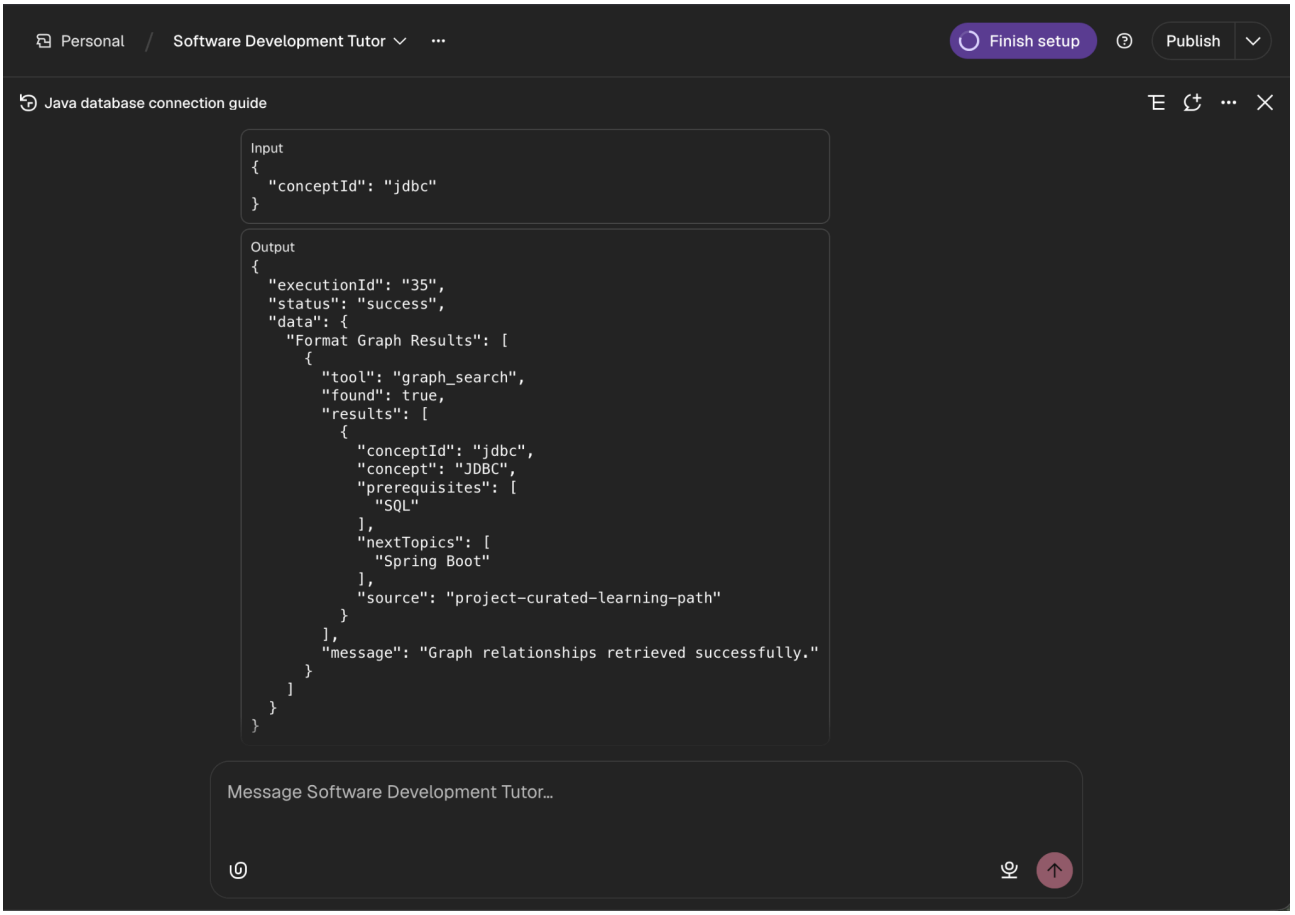
Spørgsmål: How can I connect a Java application to a relational database? Explain the process using the available learning material.

Kontrollér: Bruger agenten 02 Vector Retrieval? Finder den materialet om JDBC? Henviser den til dokumentet?




Personal
Software Development Tutor
Finish setup
Publish
Java database connection guide
"results": [
{
"chunkId": "jdbc-01",
"content": "JDBC allows Java applications to communicate with relational databases. Developers can execute SQL statements using JDBC connections and prepared statements.",
"documentId": "jdbc-guide",
"title": "Java Database Connectivity",
"source": "project-created-seed-data",
"similarity": 0.8282324,
"topics": [
"Java",
"SQL",
"JDBC"
]
},
{
"chunkId": "sql-01",
"content": "SQL is used to manage relational databases. SELECT retrieves data, INSERT adds records, UPDATE modifies records and DELETE removes records. JOIN combines related tables.",
"documentId": "sql-guide",
"title": "Introduction to SQL",
"source": "project-created-seed-data",
"similarity": 0.7113733,
"topics": [
"SQL"
]
},
{
"chunkId": "spring-01",
"content": "Spring Boot is a Java framework for
Message Software Development Tutor...


Personal
Software Development Tutor
Finish setup
Publish
Java database connection guide
]
},
{
"chunkId": "spring-01",
"content": "Spring Boot is a Java framework for building web applications and REST APIs. It provides dependency injection, automatic configuration and integration with databases.",
"documentId": "spring-guide",
"title": "Spring Boot Fundamentals",
"source": "project-created-seed-data",
"similarity": 0.67443657,
"topics": [
"Java",
"REST API",
"JDBC",
"Spring Boot"
]
}
]
}
]
}
}
03 graph retrieval
Input
{
"conceptId": "jdbc"
}
Output
Message Software Development Tutor...



Answer:

Great news — I found relevant material from both the educational documents and the knowledge graph! Here's a comprehensive explanation:

🔌 Connecting a Java Application to a Relational Database

1. The Key Technology: JDBC

JDBC (Java Database Connectivity) is the standard Java API that enables Java applications to communicate with relational databases. It acts as a bridge between your Java code and the database, allowing you to:

- **Open a connection** to a database
- **Execute SQL statements** (via connections and prepared statements)

- **Process the results** returned from the database

📄 *Source: "Java Database Connectivity" — chunk `jdbc-01`, from `project-created-seed-data`*

---

2. The Core Process (Step by Step)

Here's how the connection process works in practice:

Step 1️⃣ — Learn SQL First

Before connecting Java to a database, you need to understand **SQL** — the language used to interact with relational databases:

- `SELECT` → retrieves data
- `INSERT` → adds records
- `UPDATE` → modifies records
- `DELETE` → removes records
- `JOIN` → combines data from related tables

📄 *Source: "Introduction to SQL" — chunk `sql-01`*

Step 2️⃣ — Use JDBC to Connect

With JDBC, your Java application can:

1. **Load the JDBC driver** for your specific database (e.g., MySQL, PostgreSQL)
2. **Establish a connection** using a connection URL, username, and password
3. **Create a Statement or PreparedStatement** to execute SQL queries
4. **Process the ResultSet** to read query results
5. **Close the connection** to free resources

A simple conceptual example:

```
None
// 1. Connect to the database
Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",
"user", "password");

// 2. Prepare and execute a SQL statement
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users
WHERE id = ?");
stmt.setInt(1, 42);
```

```
// 3. Process the results
ResultSet rs = stmt.executeQuery();
while (rs.next()) {
    System.out.println(rs.getString("name"));
}

// 4. Close resources
conn.close();
```

---

3. 📚 Recommended Learning Path

The knowledge graph confirms this exact progression for you:

```
SQL  ──▸  JDBC  ──▸  Spring Boot
```

Step	Topic	Why
1️⃣	**SQL**	Prerequisite — you must understand SQL to write database queries
2️⃣	**JDBC**	Core topic — the API that connects Java to databases
3️⃣	**Spring Boot**	Next step — simplifies database integration with automatic configuration and dependency injection

Once you're comfortable with JDBC, **Spring Boot** provides higher-level abstractions (like Spring Data JPA) that make database integration even easier, handling much of the boilerplate JDBC code for you.

🗂️ *Source: `project-curated-learning-path` (graph relationships)*

---

4. Sources Used

Source Type	Document	Chunk
📄 Vector Search	Java Database Connectivity	`jdbc-01`

📄 Vector Search	Introduction to SQL	`sql-01`
📄 Vector Search	Spring Boot Fundamentals	`spring-01`
🗂️ Graph Search	Learning path: SQL → JDBC → Spring Boot	`project-curated-learning-path`

In summary: Start by mastering SQL, then use JDBC to programmatically connect your Java app to a database, and finally explore Spring Boot to handle database connections more efficiently in real-world applications. Would you like a deeper dive into any of these topics? 🚀